

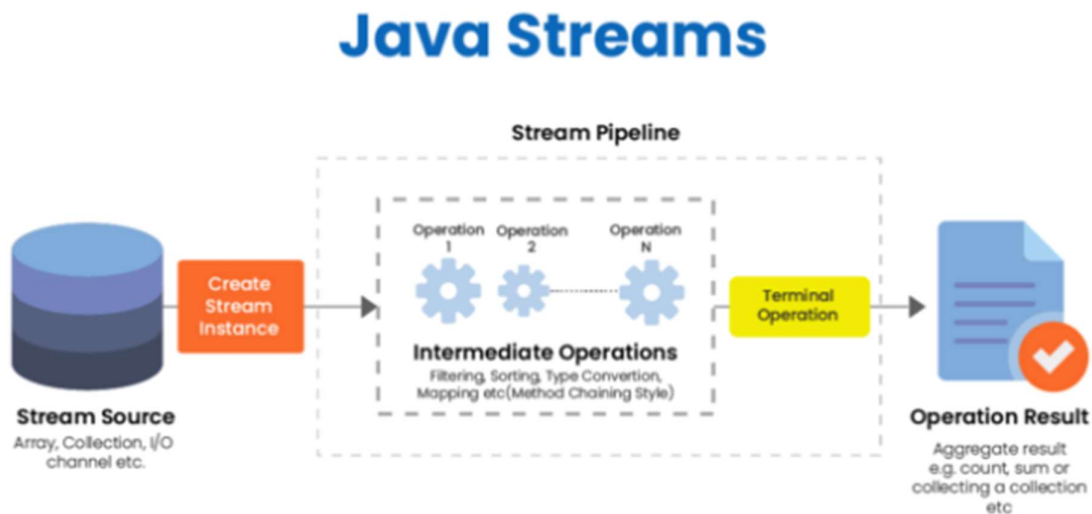
Stream in Java (Java 8)

Introduction:

- **Stream API** was introduced in **Java 8**.
- A Stream is a **sequence of elements** that supports functional operations.
- Used to process collections like list, set etc ...

Uses of Stream API

- Process collections in a clean way
- Perform operations like:
 - Filtering
 - Mapping
 - Sorting
 - Reducing
- Supports **pipelining** of multiple operations



Stream Declaration Syntax :

```
Stream<T> stream;
```

T can be any datatype or object type.

Features of Java Streams

- Stream is **not a data structure**
- Takes input from Collections, Arrays, I/O channels
- Does **not modify original data**
- Supports chaining of methods
- Uses **Lazy Evaluation** for intermediate operations
- Terminal operation produces final result

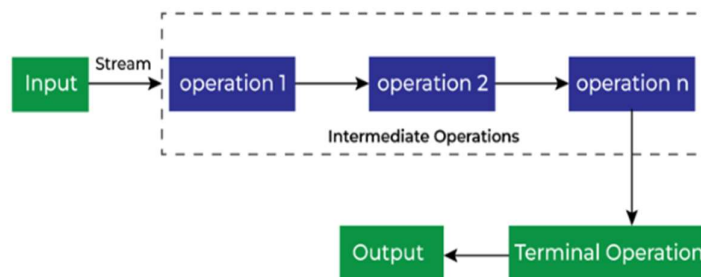
Types of Stream Operations

1) Intermediate Operations

2) Terminal Operations

Intermediate Operations

- Return another Stream
- Executed only when terminal operation is called



Characteristics

- Methods can be chained
- Transform one stream into another
- Lazy in nature

Important Intermediate Methods

1. map()

- Transforms each element

```
stream.map(x -> x * 2);
```

2. filter()

- Select elements based on condition

```
stream.filter(x -> x > 10);
```

3. sorted()

- Sorts the stream

```
stream.sorted();
```

4. flatMap()

- Flattens nested streams

```
stream.flatMap(List::stream);
```

5. distinct()

- Removes duplicates

```
stream.distinct();
```

6. peek()

- Performs action without modifying stream

```
stream.peek(System.out::println);
```

```

import java.util.*;
import java.util.stream.*;

public class IntermediateExample {
    public static void main(String[] args) {

        List<String> names = Arrays.asList(
            "Reflection", "Collection", "Stream",
            "Structure", "Sorting", "State", "Stream"
        );

        List<String> result = names.stream()

            // flatMap not needed here (used for nested lists)

            .filter(n -> n.startsWith("S")) // filter()

            .map(String::toUpperCase) // map()

            .distinct() // distinct()

            .sorted() // sorted()

            .peek(x -> System.out.println("Processing: " + x)) // peek()

            .collect(Collectors.toList()); // terminal collect()

        System.out.println("\nFinal Output:");
        System.out.println(result);
    }
}

```

Output:

```

Processing: SORTING
Processing: STATE
Processing: STREAM
Processing: STRUCTURE

```

Final Output:

```
[SORTING, STATE, STREAM, STRUCTURE]
```

Terminal Operations

- Produce final result
- End the stream pipeline

Important Terminal Methods

1. collect()

- Collect result into List/Set

```
stream.collect(Collectors.toList());
```

2. forEach()

- Iterates elements

```
stream.forEach(System.out::println);
```

3. reduce()

- Combines elements into single value

```
stream.reduce((a,b) -> a+b);
```

4. count()

- Returns number of elements

```
stream.count();
```

5. findFirst()

- Returns first element

```
stream.findFirst();
```

6. allMatch()

- Checks if all elements satisfy condition

```
stream.allMatch(x -> x > 0);
```

7. anyMatch()

- Checks if any element satisfies condition

`stream.anyMatch(x -> x == 5);`

Example :

```
import java.util.*;
import java.util.stream.*;

public class TerminalExample {
    public static void main(String[] args) {

        List<Integer> nums = Arrays.asList(2, 4, 6, 8, 10);

        // 1. forEach()
        System.out.println("forEach:");
        nums.stream().forEach(System.out::println);

        // 2. collect()
        List<Integer> evenList = nums.stream()
            .filter(x -> x % 2 == 0)
            .collect(Collectors.toList());
        System.out.println("\ncollect:");
        System.out.println(evenList);

        // 3. reduce()
        int sum = nums.stream()
            .reduce(0, (a, b) -> a + b);
        System.out.println("\nreduce (Sum): " + sum);

        // 4. count()
        long count = nums.stream().count();
        System.out.println("\ncount: " + count);

        // 5. findFirst()
        Optional<Integer> first = nums.stream().findFirst();
        System.out.println("\nfindFirst: " + first.get());

        // 6. allMatch()
        boolean allEven = nums.stream().allMatch(x -> x % 2 == 0);
        System.out.println("\nallMatch (all even?): " + allEven);

        // 7. anyMatch()
        boolean anyGreater = nums.stream().anyMatch(x -> x > 5);
        System.out.println("\nanyMatch (any >5?): " + anyGreater);
    }
}
```

Output:

forEach:

2
4
6
8
10

collect:

[2, 4, 6, 8, 10]

reduce (Sum): 30

count: 5

findFirst: 2

allMatch (all even?): true

anyMatch (any >5?): true

Benefits of Java Streams

- No extra storage needed
- Supports function pipelines
- Lazy execution improves performance
- Can be infinite streams
- Supports parallel processing
- Easy operations on large data

Real-World Use Cases

Streams are used for:

- Data processing
- JSON/XML handling
- Database operations
- Concurrent applications
- Stream-based filtering & sorting